ALLIANCE UNIVERSITY · BCA (AI/ML) · DBMS ASSIGNMENT SERIES

09

ASSIGNMENT 9

Advanced SQL Using Joins and Subqueries

Online Food Delivery System

SUBMITTED BY
Vimal Kumar Rao

PROGRAMME
BCA (AI/ML)

INSTITUTION
Alliance University

SUBMISSION DATE
24 October 2026

IN THIS ASSIGNMENT

01 Join entities

02 Use subqueries

03 Build reports

04 Interpret results

A consistent relational design carried forward from Assignments 1 and 2.

1 Objective and Brief Coverage

Real reports rarely come from one table. This assignment answers business questions about orders, customers, restaurants, payments and delivery by **joining** related tables and by nesting **subqueries** (scalar, IN, correlated, NOT EXISTS and derived tables). Eight required reports are followed by seven bonus reports.

8 + 7	5	4	Rs 549
required + bonus reports	tables joined at most (Q2)	subquery styles used	average paid spend (Q4 benchmark)

	Brief requirement	Where it is covered
✓	**Appropriate joins**	Q1 (INNER + LEFT), Q2 (5-table), Q3, Q4, Q6, Q10 to Q14
✓	**Subqueries**	Scalar Q8 | derived table Q4, Q9 | NOT EXISTS Q5 | correlated Q7, Q12 | IN Q13
✓	**Meaningful reports**	Order overview, sales ranking, customer value, payment follow-up, workload, city revenue
✓	**Multiple tables, summary statistics, related records**	Q2 (related lines), Q3 and Q4 (SUM / AVG), Q9 (average basket), Q14
✓	**SQL, output and explanation**	Every query block: SQL, OUTPUT table, "Reading the result"

Choosing the right join

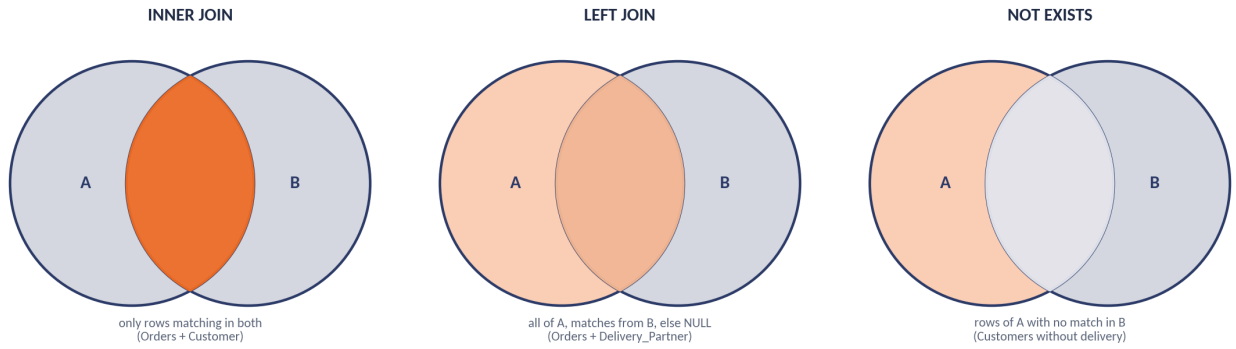


Figure 1. Join and anti-join patterns used in this assignment. Orange marks the rows returned.

Query map

#	Report	Technique	Business question
Q1	Order overview	JOIN + LEFT JOIN	Optional relationships (delivery, payment)
Q2	Order-line detail	5-table JOIN	Resolving the many-to-many Order_Item link
Q3	Top restaurants	JOIN + GROUP BY + LIMIT	Ranking by paid value
Q4	Above-average spenders	Derived table + HAVING	Compare a group with an overall benchmark
Q5	No delivered orders	NOT EXISTS	Find what is missing
Q6	Payment follow-up	JOIN + IN	Operational worklist
Q7	Latest order per customer	Correlated subquery	Per-row comparison (MAX)
Q8	Menu size and price	Scalar subqueries	Per-row aggregate in SELECT

2 Required Reports (Q1 to Q8)

Q1 Multi-table order overview

JOIN | LEFT JOIN | COALESCE

Joins Orders to Customer and Restaurant (every order has both) and **LEFT JOINs** Delivery_Partner and Payment, which are optional. COALESCE replaces the NULL partner with a readable label.

```
SELECT o.Order_ID, c.Name AS Customer, r.Restaurant_Name,
       o.Order_Status, p.Payment_Method, p.Payment_Status,
       COALESCE(d.Name, 'Not assigned') AS Delivery_Partner,
       o.Order_Date
FROM   Orders o
JOIN   Customer c ON c.Customer_ID   = o.Customer_ID
JOIN   Restaurant r ON r.Restaurant_ID = o.Restaurant_ID
LEFT JOIN Delivery_Partner d ON d.Delivery_ID = o.Delivery_ID
LEFT JOIN Payment p ON p.Order_ID = o.Order_ID
ORDER BY o.Order_Date DESC, o.Order_ID DESC;
```

OUTPUT 12 rows

Order_ID	Customer	Restaurant_Name	Order_Status	Payment_Method	Payment_Status	Delivery_Partner	Order_Date
10	Priya Sharma	Wok Express	Placed	UPI	Failed	Not assigned	2026-10-24 12:30:00
9	Nisha Rao	Green Bowl Cafe	Confirmed	Wallet	Pending	Sneha Rao	2026-10-24 12:00:00
8	Rahul Nair	Dosa Junction	Delivered	Cash	Paid	Imran Khan	2026-10-24 08:45:00
7	Kabir Das	Urban Pizza Co.	Placed	UPI	Pending	Not assigned	2026-10-23 19:00:00
6	Meera Iyer	Spice Route Kitchen	Preparing	Card	Pending	Kavya Patel	2026-10-23 13:30:00
5	Arjun Reddy	Wok Express	Out for Delivery	UPI	Pending	Sneha Rao	2026-10-23 12:00:00
4	Asha Menon	Dosa Junction	Delivered	Cash	Paid	Rohan Singh	2026-10-22 20:00:00
3	Priya Sharma	Urban Pizza Co.	Delivered	UPI	Paid	Imran Khan	2026-10-22 13:20:00
2	Rahul Nair	Green Bowl Cafe	Delivered	Card	Paid	Kavya Patel	2026-10-21 19:05:00
1	Asha Menon	Spice Route Kitchen	Delivered	UPI	Paid	Rohan Singh	2026-10-21 12:15:00
11	Arjun Reddy	Urban Pizza Co.	Delivered	Card	Paid	Rohan Singh	2026-10-20 20:00:00
12	Meera Iyer	Spice Route Kitchen	Cancelled	UPI	Refunded	Kavya Patel	2026-10-20 12:10:00

Reading the result

All 12 orders appear. Orders 7 and 10 show **Not assigned** because their Delivery_ID is NULL; an INNER JOIN would have silently dropped them.

Q2 Order-line detail

5-table JOIN | calculated column

Connects order headers, customers, restaurants, order lines and menu items. Line_Subtotal is computed as Quantity x Unit_Price using the stored historical price.

```
SELECT o.Order_ID, c.Name AS Customer, r.Restaurant_Name,
       mi.Item_Name, oi.Quantity, oi.Unit_Price,
       (oi.Quantity * oi.Unit_Price) AS Line_Subtotal
FROM   Orders o
JOIN   Customer c    ON c.Customer_ID   = o.Customer_ID
JOIN   Restaurant r  ON r.Restaurant_ID = o.Restaurant_ID
JOIN   Order_Item oi ON oi.Order_ID     = o.Order_ID
JOIN   Menu_Item mi  ON mi.Item_ID      = oi.Item_ID
ORDER BY o.Order_ID, mi.Item_Name;
```

OUTPUT 21 rows

Order_ID	Customer	Restaurant_Name	Item_Name	Quantity	Unit_Price	Line_Subtotal
1	Asha Menon	Spice Route Kitchen	Chicken Biryani	2	220	440
1	Asha Menon	Spice Route Kitchen	Vegetable Meals	1	150	150
2	Rahul Nair	Green Bowl Cafe	Lemon Iced Tea	2	70	140
2	Rahul Nair	Green Bowl Cafe	Veg Buddha Bowl	1	160	160
3	Priya Sharma	Urban Pizza Co.	Garlic Bread	1	119	119
3	Priya Sharma	Urban Pizza Co.	Margherita Pizza	1	249	249
4	Asha Menon	Dosa Junction	Filter Coffee	2	40	80
4	Asha Menon	Dosa Junction	Masala Dosa	2	90	180
5	Arjun Reddy	Wok Express	Spring Rolls	1	120	120
5	Arjun Reddy	Wok Express	Veg Hakka Noodles	2	180	360
6	Meera Iyer	Spice Route Kitchen	Chicken Biryani	1	220	220
6	Meera Iyer	Spice Route Kitchen	Paneer Biryani	1	180	180
7	Kabir Das	Urban Pizza Co.	Farmhouse Pizza	1	329	329
8	Rahul Nair	Dosa Junction	Filter Coffee	1	40	40
8	Rahul Nair	Dosa Junction	Ghee Roast Dosa	1	140	140
9	Nisha Rao	Green Bowl Cafe	Paneer Wrap	2	130	260
9	Nisha Rao	Green Bowl Cafe	Veg Buddha Bowl	1	160	160
10	Priya Sharma	Wok Express	Chilli Paneer	1	220	220
10	Priya Sharma	Wok Express	Veg Hakka Noodles	1	180	180
11	Arjun Reddy	Urban Pizza Co.	Margherita Pizza	2	249	498
12	Meera Iyer	Spice Route Kitchen	Vegetable Meals	1	150	150

Reading the result

21 rows, one per order line, matching the 21 rows loaded in Assignment 7. Joining through Order_Item is what resolves the many-to-many relationship between Orders and Menu_Item.

Q3 Top restaurants by paid line value

JOIN | SUM | GROUP BY | LIMIT

Ranks restaurants by the value of order lines whose payment is **Paid**. The Paid condition sits in the join so that only paid orders contribute; cancelled orders are excluded in WHERE.

```
SELECT r.Restaurant_Name,
       SUM(oi.Quantity * oi.Unit_Price) AS Paid_Line_Value
FROM   Restaurant r
JOIN   Orders o     ON o.Restaurant_ID = r.Restaurant_ID
JOIN   Order_Item oi ON oi.Order_ID    = o.Order_ID
JOIN   Payment p    ON p.Order_ID = o.Order_ID
                   AND p.Payment_Status = 'Paid'
WHERE  o.Order_Status <> 'Cancelled'
GROUP BY r.Restaurant_ID, r.Restaurant_Name
ORDER BY Paid_Line_Value DESC, r.Restaurant_Name ASC
LIMIT 3;
```

OUTPUT 3 rows

Restaurant_Name	Paid_Line_Value
Urban Pizza Co.	866
Spice Route Kitchen	590
Dosa Junction	440

Reading the result

Urban Pizza Co. leads with Rs 866 (orders 3 and 11). Only the top three are shown (LIMIT 3). Green Bowl Cafe (Rs 300) ranks fourth, and Wok Express has no Paid order at all, so it never enters the join.

Q4 Above-average customer paid spending

derived table | subquery in HAVING | AVG | SUM

A **derived table** (the innermost subquery) first totals paid spend per customer; AVG of those totals gives the benchmark, and HAVING keeps customers above it.

```
SELECT c.Customer_ID, c.Name,
       SUM(oi.Quantity * oi.Unit_Price) AS Paid_Spend
FROM   Customer c
JOIN   Orders o ON o.Customer_ID = c.Customer_ID
JOIN   Payment p ON p.Order_ID = o.Order_ID AND p.Payment_Status = 'Paid'
JOIN   Order_Item oi ON oi.Order_ID = o.Order_ID
GROUP BY c.Customer_ID, c.Name
HAVING SUM(oi.Quantity * oi.Unit_Price) > (
    SELECT AVG(customer_spending.Paid_Spend)
    FROM (
        SELECT o2.Customer_ID,
               SUM(oi2.Quantity * oi2.Unit_Price) AS Paid_Spend
        FROM   Orders o2
        JOIN   Payment p2 ON p2.Order_ID = o2.Order_ID
                         AND p2.Payment_Status = 'Paid'
        JOIN   Order_Item oi2 ON oi2.Order_ID = o2.Order_ID
        GROUP BY o2.Customer_ID
    ) AS customer_spending
)
ORDER BY Paid_Spend DESC;
```

OUTPUT 1 row

Customer_ID	Name	Paid_Spend
1	Asha Menon	850

Reading the result

Four customers have paid orders: 850, 498, 480 and 368. Their average is **Rs 549**, and only Asha Menon (Rs 850) is above it. The inner query is evaluated once; it is not correlated.

Q5 Customers with no delivered orders

NOT EXISTS | correlated subquery

An **anti-join**: returns customers for whom *no* Delivered order exists. NOT EXISTS stops searching as soon as one match is found.

```
SELECT c.Customer_ID, c.Name
FROM   Customer c
WHERE  NOT EXISTS (
    SELECT 1
    FROM   Orders o
    WHERE  o.Customer_ID  = c.Customer_ID
      AND  o.Order_Status = 'Delivered'
)
ORDER BY c.Customer_ID;
```

OUTPUT 3 rows

Customer_ID	Name
5	Meera Iyer
6	Kabir Das
7	Nisha Rao

Reading the result

Meera Iyer (one preparing, one cancelled order), Kabir Das (placed) and Nisha Rao (confirmed) have never received a delivery. This is the list a retention team would contact first. Unlike NOT IN, NOT EXISTS stays correct even if a NULL appears in the subquery.

Q6 Pending / failed payment follow-up

JOIN | IN list | operational report

Lists orders whose payment is Pending or Failed, with customer and restaurant, newest first, so staff know whom to chase.

```
SELECT o.Order_ID, c.Name AS Customer, r.Restaurant_Name,
       o.Order_Status, p.Payment_Method, p.Payment_Status,
       o.Order_Date
FROM   Orders o
JOIN   Customer c   ON c.Customer_ID   = o.Customer_ID
JOIN   Restaurant r ON r.Restaurant_ID = o.Restaurant_ID
JOIN   Payment p    ON p.Order_ID      = o.Order_ID
WHERE  p.Payment_Status IN ('Pending', 'Failed')
ORDER BY o.Order_Date DESC;
```

OUTPUT 5 rows

Order_ID	Customer	Restaurant_Name	Order_Status	Payment_Method	Payment_Status	Order_Date
10	Priya Sharma	Wok Express	Placed	UPI	Failed	2026-10-24 12:30:00
9	Nisha Rao	Green Bowl Cafe	Confirmed	Wallet	Pending	2026-10-24 12:00:00
7	Kabir Das	Urban Pizza Co.	Placed	UPI	Pending	2026-10-23 19:00:00
6	Meera Iyer	Spice Route Kitchen	Preparing	Card	Pending	2026-10-23 13:30:00
5	Arjun Reddy	Wok Express	Out for Delivery	UPI	Pending	2026-10-23 12:00:00

Reading the result

Five orders need attention. Order 10 is the most urgent: its payment **Failed** but the order was still placed. Order 5 is already Out for Delivery while its UPI payment is still Pending, which is worth checking.

Q7 Latest order per customer

correlated subquery | MAX

A **correlated subquery** finds each customer's most recent order date, and the outer query keeps the order that has exactly that date.

```
SELECT c.Customer_ID, c.Name,
       o.Order_ID, o.Order_Date, o.Order_Status
FROM   Customer c
JOIN   Orders o ON o.Customer_ID = c.Customer_ID
WHERE  o.Order_Date = (
    SELECT MAX(o2.Order_Date)
    FROM   Orders o2
    WHERE  o2.Customer_ID = c.Customer_ID   -- re-evaluated per customer
)
ORDER BY c.Customer_ID;
```

OUTPUT 7 rows

Customer_ID	Name	Order_ID	Order_Date	Order_Status
1	Asha Menon	4	2026-10-22 20:00:00	Delivered
2	Rahul Nair	8	2026-10-24 08:45:00	Delivered
3	Priya Sharma	10	2026-10-24 12:30:00	Placed
4	Arjun Reddy	5	2026-10-23 12:00:00	Out for Delivery
5	Meera Iyer	6	2026-10-23 13:30:00	Preparing
6	Kabir Das	7	2026-10-23 19:00:00	Placed
7	Nisha Rao	9	2026-10-24 12:00:00	Confirmed

Reading the result

One row per customer. If a customer had two orders with an identical latest timestamp, both would be returned; that is intentional here.

Q8 Menu size and average menu price

scalar subquery | COUNT | AVG | ROUND

Two **scalar subqueries** in the SELECT list compute each restaurant's menu size and average current price.

```
SELECT r.Restaurant_Name,
       (SELECT COUNT(*)
        FROM   Menu_Item mi
        WHERE  mi.Restaurant_ID = r.Restaurant_ID)          AS Menu_Items,
       (SELECT ROUND(AVG(mi.Price), 2)
        FROM   Menu_Item mi
        WHERE  mi.Restaurant_ID = r.Restaurant_ID)          AS Average_Menu_Price
FROM   Restaurant r
ORDER BY Menu_Items DESC, r.Restaurant_Name;
```

OUTPUT 5 rows

Restaurant_Name	Menu_Items	Average_Menu_Price
Dosa Junction	3	90
Green Bowl Cafe	3	120
Spice Route Kitchen	3	183.33
Urban Pizza Co.	3	232.33
Wok Express	3	173.33

Reading the result

Every restaurant lists three items. Urban Pizza Co. is the premium option (average Rs 232.33) and Dosa Junction the cheapest (Rs 90.00). These are *current* menu prices, not historical order prices.

3 Visual Summary

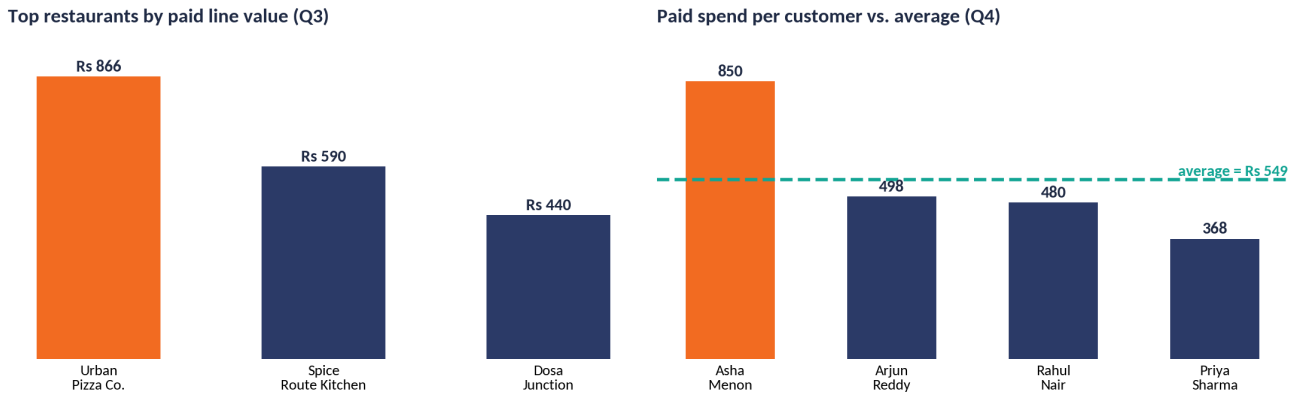


Figure 2. Left: paid line value of the top three restaurants (Q3). Right: paid spend per customer against the Rs 549 average (Q4).

Insight

Revenue is concentrated: Urban Pizza Co. alone accounts for Rs 866 of the Rs 2,196 delivered value (about 39%), and one customer (Asha Menon) accounts for Rs 850 of the Rs 2,196 paid in total (about 39% as well). With only 12 fictional orders this is a demonstration, not a business conclusion.

4 Bonus Reports (Q9 to Q15)

Seven further reports use the same schema to show derived tables in FROM, LEFT JOIN with aggregation, IN subqueries and a data-quality audit.

Q9 BONUS: average order value per restaurant

derived table in FROM | AVG | JOIN

A derived table first totals each order, then the outer query averages those totals per restaurant. Averaging line prices directly would give the wrong answer.

```
SELECT r.Restaurant_Name,
       COUNT(*)                          AS Orders,
       ROUND(AVG(ot.Order_Total), 2)     AS Avg_Order_Value
FROM   Restaurant r
JOIN   Orders o ON o.Restaurant_ID = r.Restaurant_ID
JOIN  (SELECT Order_ID, SUM(Quantity * Unit_Price) AS Order_Total
       FROM   Order_Item
       GROUP BY Order_ID) AS ot ON ot.Order_ID = o.Order_ID
WHERE  o.Order_Status <> 'Cancelled'
GROUP BY r.Restaurant_ID, r.Restaurant_Name
ORDER BY Avg_Order_Value DESC;
```

OUTPUT 5 rows

Restaurant_Name	Orders	Avg_Order_Value
Spice Route Kitchen	2	495
Wok Express	2	440
Urban Pizza Co.	3	398.33
Green Bowl Cafe	2	360
Dosa Junction	2	220

Reading the result

Spice Route Kitchen has the highest average basket (Rs 495 across 2 orders) and Dosa Junction the lowest (Rs 220). Urban Pizza Co. has the most orders (3) but a lower average (Rs 398.33).

Q10 BONUS: delivery partner workload

LEFT JOIN | GROUP BY | COUNT

Counts assigned orders per delivery partner, keeping partners with none.

```
SELECT d.Name, d.Status,
       COUNT(o.Order_ID) AS Assigned_Orders
FROM   Delivery_Partner d
LEFT JOIN Orders o ON o.Delivery_ID = d.Delivery_ID
GROUP BY d.Delivery_ID, d.Name, d.Status
ORDER BY Assigned_Orders DESC, d.Name;
```

OUTPUT 4 rows

Delivery_Partner	Status	Assigned_Orders
Kavya Patel	Available	3
Rohan Singh	Busy	3
Imran Khan	Busy	2
Sneha Rao	Offline	2

Reading the result

Sneha Rao is marked **Offline** yet is assigned 2 orders (5 and 9), both still active. A real system should reject assignments to offline partners.

Q11 BONUS: best-selling items by quantity

JOIN | SUM | LIMIT

Ranks menu items by units sold in non-cancelled orders, using a name tie-breaker.

```
SELECT mi.Item_Name, r.Restaurant_Name,
       SUM(oi.Quantity) AS Units_Sold
FROM   Menu_Item mi
JOIN   Restaurant r  ON r.Restaurant_ID = mi.Restaurant_ID
JOIN   Order_Item oi ON oi.Item_ID      = mi.Item_ID
JOIN   Orders o      ON o.Order_ID      = oi.Order_ID
WHERE  o.Order_Status <> 'Cancelled'
GROUP BY mi.Item_ID, mi.Item_Name, r.Restaurant_Name
ORDER BY Units_Sold DESC, mi.Item_Name
LIMIT 5;
```

OUTPUT 5 rows

Item_Name	Restaurant_Name	Units_Sold
Chicken Biryani	Spice Route Kitchen	3
Filter Coffee	Dosa Junction	3
Margherita Pizza	Urban Pizza Co.	3
Veg Hakka Noodles	Wok Express	3
Lemon Iced Tea	Green Bowl Cafe	2

Reading the result

Four items tie at 3 units; the alphabetical tie-breaker decides the order. The four tied items are the best sellers of Spice Route Kitchen, Dosa Junction, Urban Pizza Co. and Wok Express respectively.

Q12 BONUS: items priced above their restaurant's average

correlated subquery | AVG

A correlated subquery compares each item's price with the average price of *its own* restaurant.

```sql
SELECT r.Restaurant_Name, mi.Item_Name, mi.Price
FROM   Menu_Item mi
JOIN   Restaurant r ON r.Restaurant_ID = mi.Restaurant_ID
WHERE  mi.Price > (
    SELECT AVG(m2.Price)
    FROM   Menu_Item m2
    WHERE  m2.Restaurant_ID = mi.Restaurant_ID
)
ORDER BY r.Restaurant_Name, mi.Price DESC;
```

OUTPUT 8 rows

Restaurant_Name	Item_Name	Price
Dosa Junction	Ghee Roast Dosa	140
Green Bowl Cafe	Veg Buddha Bowl	160
Green Bowl Cafe	Paneer Wrap	130
Spice Route Kitchen	Chicken Biryani	220
Urban Pizza Co.	Farmhouse Pizza	329
Urban Pizza Co.	Margherita Pizza	249
Wok Express	Chilli Paneer	220
Wok Express	Veg Hakka Noodles	180

Reading the result

Eight of the fifteen items sit above their own restaurant's average. Comparing against a single global average would have produced a different, less meaningful list.

Q13 BONUS: customers who ordered from Urban Pizza Co.

IN subquery JOIN

An **IN** subquery: the inner query returns customer IDs who ordered from one named restaurant.

```
SELECT c.Customer_ID, c.Name
FROM   Customer c
WHERE  c.Customer_ID IN (
    SELECT o.Customer_ID
    FROM   Orders o
    JOIN   Restaurant r ON r.Restaurant_ID = o.Restaurant_ID
    WHERE  r.Restaurant_Name = 'Urban Pizza Co.'
)
ORDER BY c.Customer_ID;
```

OUTPUT 3 rows

Customer_ID	Name
3	Priya Sharma
4	Arjun Reddy
6	Kabir Das

Reading the result

Three customers (Priya, Arjun, Kabir) have ordered from Urban Pizza Co. A customer with several orders there still appears once.

Q14 BONUS: revenue by delivery city

JOIN GROUP BY SUM

Joins each order to the delivery address it used and totals non-cancelled line value per city.

```
SELECT a.City,
       COUNT(DISTINCT o.Order_ID)        AS Orders,
       SUM(oi.Quantity * oi.Unit_Price)  AS Line_Value
FROM   Orders o
JOIN   Address a     ON a.Address_ID = o.Address_ID
JOIN   Order_Item oi ON oi.Order_ID  = o.Order_ID
WHERE  o.Order_Status <> 'Cancelled'
GROUP BY a.City
ORDER BY Line_Value DESC;
```

OUTPUT 3 rows

City	Orders	Line_Value
Bengaluru	5	2038
Hosur	5	1787
Chennai	1	400

Reading the result

Bengaluru and Hosur each receive five orders; Chennai only one. The totals add up to Rs 4,225, matching Assignment 8, Q5.

Q15 BONUS: integrity audit, order lines from the wrong restaurant

JOIN | data-quality check

The schema cannot stop an order for restaurant A from containing a menu item belonging to restaurant B. This query audits for that mistake.

```
SELECT o.Order_ID, o.Restaurant_ID AS Order_Restaurant,
       mi.Item_ID, mi.Restaurant_ID  AS Item_Restaurant
FROM   Orders o
JOIN   Order_Item oi ON oi.Order_ID = o.Order_ID
JOIN   Menu_Item mi  ON mi.Item_ID  = oi.Item_ID
WHERE  mi.Restaurant_ID <> o.Restaurant_ID;
```

OUTPUT 0 rows (empty set = no problems found)

Order_ID	Order_Restaurant	Item_ID	Item_Restaurant

Reading the result

0 rows is the healthy result: every order line belongs to the restaurant on its order. Assignment 3 proposes a composite key (or trigger) so the database enforces this rule instead of relying on an audit.

5 Key Observations and Limits

- **LEFT JOIN vs JOIN (Q1).** Delivery and payment are optional during an order's life; an INNER JOIN would hide orders 7 and 10 from the overview.
- **Paid line value is not profit (Q3).** The schema has no tax, delivery fee, discount or refund amounts, so these figures are line-item value only.
- **Results depend on the data (Q4).** Asha Menon is the only customer above average in *this* fictional dataset.
- **NOT EXISTS vs NOT IN (Q5).** NOT EXISTS stays correct when the subquery can return NULL; NOT IN does not.
- **Ties (Q7).** A customer with two orders at the identical latest timestamp would return both rows.
- **Rules the schema cannot enforce (Q10, Q15).** Nothing stops an Offline partner being assigned or a line from another restaurant being added. Triggers, stored procedures or a composite foreign key (proposed in Assignment 3) would close both gaps.

6 Execution Steps

- Run Assignments 5, 6 and 7 (or the combined **Complete_Online_Food_Delivery_Setup.sql**) on an empty database.
- Open **Assignment_9_Advanced_SQL_Queries.sql** in MySQL Workbench and run each SELECT, then compare with the OUTPUT tables.
- The OUTPUT tables were produced by executing the SQL printed here against the exact setup-script rows in a local SQLite validation harness, because no MySQL server was available. The syntax is standard and accepted by MySQL 8; capture Workbench screenshots from your own server as final evidence.

7 Conclusion

The reports show how a normalised schema answers operational and analytical questions: joins reassemble facts that were deliberately stored once, aggregate subqueries compare groups with benchmarks, and anti-joins find what is missing. Reliable output still depends on enforced constraints and clearly stated business definitions.